

# Calculus for Machine Learning

*My revision notes, built for understanding not memorizing*

## 1. Why calculus matters for ML

---

Every ML model starts out wrong. It makes bad guesses, and then it needs a way to figure out exactly how to become less wrong, step by step. Calculus is the math of change, it tells you how a small tweak in one number affects another number. That is exactly the tool a model needs to improve itself.

When a neural network learns, it is not doing anything mystical, it is repeatedly asking one calculus question over and over: if I nudge this weight a tiny bit, does my error go up or down, and by how much. That single question, asked millions of times, is literally what training a model means. Without calculus, there is no way to answer it.

## 2. Functions, quick refresher

---

A function is just a machine, you put a number in, a rule processes it, and one number comes out. In ML, a model is one giant function, it takes in data and spits out a prediction. Calculus is entirely about studying how these input-output machines behave, especially how the output reacts when you wiggle the input.

- **Example:**  $f(x) = x^2$ . Put in 3, get 9. Put in 4, get 16. Calculus asks, if I move  $x$  from 3 to 3.01, how much does the output actually move.

## 3. Limits

---

Here is the actual problem limits solve. Sometimes you want to know what a function is doing at a point where you cannot just plug in the number directly, either because the function is undefined there, or because you want the behavior at an infinitely small zoom level, not at one fixed spot. A limit lets you ask what value a function is heading toward, as you get closer and closer to a point, without needing to actually land exactly on it.

### Notation

$$\lim (x \rightarrow a) f(x) = L$$

Read as: as  $x$  gets closer and closer to  $a$ ,  $f(x)$  gets closer and closer to  $L$ .

- **Example:**  $f(x) = (x^2 - 1)/(x - 1)$  is technically undefined at  $x = 1$ , since you would divide by zero. But as  $x$  creeps toward 1 from both sides,  $f(x)$  creeps toward 2. The limit exists and equals 2, even though the function itself has a hole at that exact point.

Why this matters for ML: the entire idea of a derivative, covered next, is literally built by taking a limit. Without limits, you cannot even define what an instantaneous rate of change means, you could only ever talk about average change over a chunk of distance.

## 4. Continuity

---

A function is continuous if you can draw its entire graph without ever lifting your pen. No sudden jumps, no holes, no breaks. This matters because most of the clean calculus tools, especially derivatives, only reliably work on functions that behave continuously, a function with a sudden jump does not have a well defined slope right at that jump.

- **Example:** a step function that jumps from 0 to 1 the moment  $x$  crosses zero is not continuous there, and asking for its slope exactly at that jump point does not make sense.

For ML, this is why activation functions are chosen carefully. A perfectly sharp step function is a bad activation function to learn with, because it has no useful slope for the model to learn from at the jump. Smoother functions like sigmoid or ReLU (mostly smooth, with one sharp corner) are much friendlier for the training process.

## 5. Derivatives, the core idea

---

This is the single most important concept in this entire note. A derivative answers one question: if I nudge the input by a tiny amount, how much does the output change, right at this exact instant. It is the instantaneous slope of a function at one specific point.

Why not just use average rate of change instead? Because average rate of change only tells you the overall trend across a big stretch, it hides everything happening in between. If you are driving and want to know your speed right now, at this exact second, not your average speed over the whole trip, you need the instantaneous version. That is exactly the gap a derivative fills.

### Definition

$$f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$$

*This is literally the slope formula (rise over run) squeezed down to an infinitely tiny run, using a limit to make that squeeze mathematically valid.*

## Common derivative rules you will actually use

- Power Rule: derivative of  $x^n$  is  $n$  times  $x^{(n-1)}$ . Example, derivative of  $x^3$  is  $3x^2$ .
- Constant Rule: derivative of any plain constant number is 0, since a flat line never changes.
- Sum Rule: derivative of  $(f + g)$  is  $f' + g'$ , you can differentiate term by term.
- Product Rule: derivative of  $(f \text{ times } g)$  is  $f'$  times  $g$ , plus  $f$  times  $g'$ .
- Quotient Rule: derivative of  $(f / g)$  is  $(f' g - f g') / g^2$ .

## What the derivative's sign and size actually tell you

- Positive derivative: function is going uphill, increasing.
- Negative derivative: function is going downhill, decreasing.
- Zero derivative: function is momentarily flat, a candidate for a peak, a valley, or a flat plateau.

This last point, where the derivative hits zero, is exactly how optimization works, covered in section 8, since the lowest point of an error curve is always a flat spot.

## 6. Chain Rule

Real functions are rarely simple, they are usually functions stacked inside other functions, like a Russian nesting doll. A neural network is an extreme example of this, it is layer after layer of functions feeding into each other. The chain rule is the tool that lets you find the derivative of this whole nested stack, without needing to painfully expand it all into one giant equation first.

### Formula

**if  $y = f(g(x))$ , then  $dy/dx = f'(g(x)) \times g'(x)$**

*Take the derivative of the outer function, keeping the inner function untouched inside it, then multiply by the derivative of the inner function.*

- **Example:**  $y = (3x + 1)^2$ . Treat  $(3x+1)$  as the inner piece. Outer derivative gives  $2(3x+1)$ , inner derivative gives 3. Multiply them:  $dy/dx = 6(3x+1)$ .

This is the single most important rule for machine learning specifically, because it is the exact engine behind backpropagation. A neural network's error passes back through many stacked layers, and the chain rule is what lets you compute how much each individual weight, buried deep inside all those layers, contributed to the final error, one multiplication step at a time.

## 7. Partial Derivatives

---

Real models almost never depend on just one input, a house price prediction depends on square footage, location, number of rooms, and more, all at once. A partial derivative answers: if I only nudge one of these inputs, and freeze every other input exactly where it is, how does the output change. It isolates the effect of one variable at a time.

### Notation

#### $df/dx$ (partial d, not a normal d)

*Read as: the rate of change of  $f$  with respect to  $x$  only, treating every other variable as a fixed constant while you do it.*

- **Example:**  $f(x, y) = x^2 + 3xy$ . Partial derivative with respect to  $x$  is  $2x + 3y$  (treat  $y$  as if it were just a plain number). Partial derivative with respect to  $y$  is  $3x$  (treat  $x$  as a plain number, and  $x^2$  disappears entirely since it has no  $y$  in it).

Why we need this instead of a normal derivative: a normal derivative only makes sense for a function with one single input. The moment you have multiple inputs, which is basically always true in ML, you need a way to study each input's individual effect separately, and partial derivatives are exactly that tool.

## 8. Gradient

---

Once you have a bunch of partial derivatives, one for each input variable, you can bundle them all together into a single arrow, called the gradient. This arrow points in the exact direction where the function increases the fastest, and its length tells you how steep that increase is.

### Notation

#### Gradient of $f = [ df/dx_1, df/dx_2, \dots, df/dx_n ]$

*One partial derivative per input variable, stacked into a vector.*

Why this matters enormously for ML: a model's error function usually depends on thousands, sometimes billions, of weights at once. The gradient tells you, for every single one of those weights simultaneously, which direction makes the error worse. Flip that direction around, and you know exactly which way to adjust every weight to make the error better. This single idea is the entire foundation of how models learn.

## 9. Gradient Descent

---

This is where derivatives and gradients actually pay off directly. The goal of training a model is to find the weights that make the error as small as possible. You cannot just guess, and for a model with millions of weights, you definitely cannot try every possible combination. Gradient descent is the trick: take small steps in the exact opposite direction of the gradient, since the gradient points toward increasing error, and the opposite direction points toward decreasing error.

### Update Rule

$$\text{new\_weight} = \text{old\_weight} - \text{learning\_rate} \times \text{gradient}$$

*The learning rate controls how big each step is. Too big and you overshoot the low point and bounce around, too small and training crawls painfully slowly.*

Picture standing on a foggy hillside at night, trying to reach the lowest valley. You cannot see the whole landscape, but you can feel which direction is downhill right under your feet. Take a small step that way, feel again, take another step. Repeat this thousands of times and you eventually reach the bottom. That is exactly gradient descent, and the reason it needs the gradient in the first place is because that is literally the only way to feel which direction is downhill from wherever you currently stand.

## 10. Critical Points, Maxima, Minima and Saddle Points

---

A critical point is any spot where the derivative equals zero, meaning the function is momentarily flat there. But flat could mean three very different things: the top of a hill, the bottom of a valley, or a weird flat pass-through point called a saddle. You need a second check to tell these apart.

### The Second Derivative Test

The second derivative is just the derivative of the derivative, it tells you if the slope itself is increasing or decreasing, in other words, whether the curve is bending upward like a cup, or downward like a dome.

- If the second derivative is positive at a critical point, the curve bends upward there, so it is a minimum (a valley, bottom of a cup).
- If the second derivative is negative there, the curve bends downward, so it is a maximum (a peak, top of a dome).
- If the second derivative is zero, the test is inconclusive, this often signals a saddle point, especially common in multivariable functions.

Why this matters for ML: training a model means searching for the minimum of an error function. But an error surface with many weights can have peaks, valleys, and saddle points scattered everywhere. Knowing how to tell these apart is exactly what stops an optimizer from accidentally stopping at a peak or getting confused at a flat saddle instead of a genuine valley.

## 11. Hessian Matrix

---

The second derivative test above works fine for a function with one input. But ML models have many inputs at once, so you need the multivariable version of the second derivative, and that is called the Hessian. It is simply a full grid of every possible second partial derivative, capturing how every input variable's slope changes with respect to every other input variable, all at once.

### What it looks like

**Hessian = a matrix where entry  $(i,j)$  is  $d^2f / dx_i dx_j$**

*A square grid, one row and one column per input variable.*

The Hessian tells you the curvature of the error surface in every direction simultaneously, which is exactly what advanced optimizers use to take smarter steps than plain gradient descent, since it also captures how quickly the slope itself is changing, not just the slope alone.

## 12. Convex vs Non-Convex Functions

---

This is one of the most practically important ideas for anyone training models. A convex function is shaped like a single smooth bowl, it has exactly one lowest point, and no matter where you start rolling a ball inside it, the ball always ends up at that same single true bottom.

A non-convex function is lumpy, it has multiple valleys (local minima) at different depths, and a ball rolling downhill can easily get trapped in a shallow valley that is not actually the lowest point overall (the global minimum).

- **Why this matters:** Linear Regression's error function is convex, so gradient descent is guaranteed to find the true best answer every single time. A deep neural network's error function is famously non-convex, full of bumps and valleys, so gradient descent might get stuck in a decent but not perfect valley. This is exactly why training a deep network is more of an art involving tricks like momentum and random restarts, while training a linear model is a much more predictable, guaranteed process.

## 13. Jacobian Matrix

---

The gradient (section 8) works when you have many inputs but only one single output number. But sometimes a function takes many inputs and also produces many outputs all at once, like a layer in a neural network that takes in a vector of numbers and outputs a whole new vector of numbers. The Jacobian is simply the generalization of the gradient for this many-inputs-many-outputs situation.

### What it looks like

**Jacobian = a matrix where entry  $(i,j)$  is  $d(\text{output}_i) / d(\text{input}_j)$**

*One row per output, one column per input, capturing every input's effect on every output simultaneously.*

Why we need a separate name and tool for this: a single gradient vector cannot capture multiple outputs at once, you would need a whole separate gradient for every single output, and the Jacobian is exactly that, all of those gradients stacked together neatly into one matrix. This is essential in backpropagation whenever a layer transforms one vector into another vector, which is basically every layer in a network.

## 14. Taylor Series

---

Some functions are genuinely painful or slow to compute exactly, especially deep inside a training loop that runs millions of times. The Taylor series solves a very specific problem: it lets you approximate almost any smooth function using a simple sum of easy-to-compute pieces, built entirely from that function's value and its derivatives at a single known point.

### Formula

$$f(x) \approx f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots$$

*You use the function's value, slope, and curvature at one known point  $a$ , to build a cheap approximation of the function nearby.*

Why this matters for ML: many advanced optimization algorithms, like Newton's Method, work by taking just the first couple of terms of this series (essentially using the gradient and the Hessian together) to build a smarter, curved approximation of the error surface near the current point, letting them jump toward the minimum faster than plain gradient descent, which only ever uses the straight-line slope alone.

## 15. Integrals

---

A derivative breaks a function down into its instantaneous rate of change. An integral does the exact opposite job, it builds a function back up by accumulating tiny pieces together, most commonly used to find the total area sitting under a curve.

### Notation

#### Integral of $f(x) dx$

*Reads as: sum up  $f(x)$  times an infinitely thin sliver of width  $dx$ , across the whole range, and add all those slivers together.*

Why area under a curve actually matters: think of a car's speed graph over time. The area under that speed curve, between two points in time, gives you the total distance traveled, even if the speed was constantly changing the whole time. You could never get that total by just looking at one instantaneous speed reading, you need the accumulation, and that accumulation is exactly what an integral computes.

## The Fundamental Theorem of Calculus

This is the deep result that ties derivatives and integrals together as two sides of the same coin: differentiating undoes integrating, and integrating undoes differentiating. This is precisely why you are allowed to solve integrals by finding a function whose derivative matches what you started with, instead of manually summing infinite tiny slivers by hand every single time.

## 16. Why Integrals matter for ML and Probability

---

You already know from probability that a continuous variable's PDF gives density, not a direct probability, and that you get an actual probability by finding the area under the curve between two points. That area calculation is literally an integral. Without integrals, you cannot get an actual number probability out of a continuous distribution like the normal distribution at all, you would be stuck with density values that do not directly mean anything on their own.

- **Example:** finding the probability that a randomly chosen adult's height falls between 160cm and 170cm means integrating the height's PDF curve between those two points.

Integrals also show up whenever you calculate an expected value for a continuous random variable, since summing over infinite possible values requires an integral instead of the simple sum you would use for a discrete variable.

## 17. Putting it all together, where each idea plugs into ML

---

- **Derivatives and the chain rule:** the exact mechanism behind backpropagation, figuring out how much each weight in a network contributed to the final error.
- **Gradient:** tells an optimizer which direction makes the error worse, across every weight simultaneously.
- **Gradient descent:** the actual step by step training loop that nudges weights toward lower error, using that gradient.
- **Critical points and the Hessian:** help tell apart a genuine minimum from a peak or a confusing saddle point.
- **Convexity:** tells you whether your optimizer is guaranteed to find the true best answer, or might get stuck in a decent but wrong valley.
- **Jacobian:** handles backpropagation through layers that transform whole vectors of numbers, not just single numbers.
- **Taylor series:** the basis for smarter, faster optimizers that use curvature, not just slope.
- **Integrals:** turn density into actual probability, and let you compute expected values for continuous random variables.

*End of notes. If short on time before revision, prioritize derivatives, chain rule, gradient descent and convexity, these four ideas are what actually make a model learn.*